Temporal Safety and Threat Detection for Software-Defined Vehicles

The whole project explained simply, with code

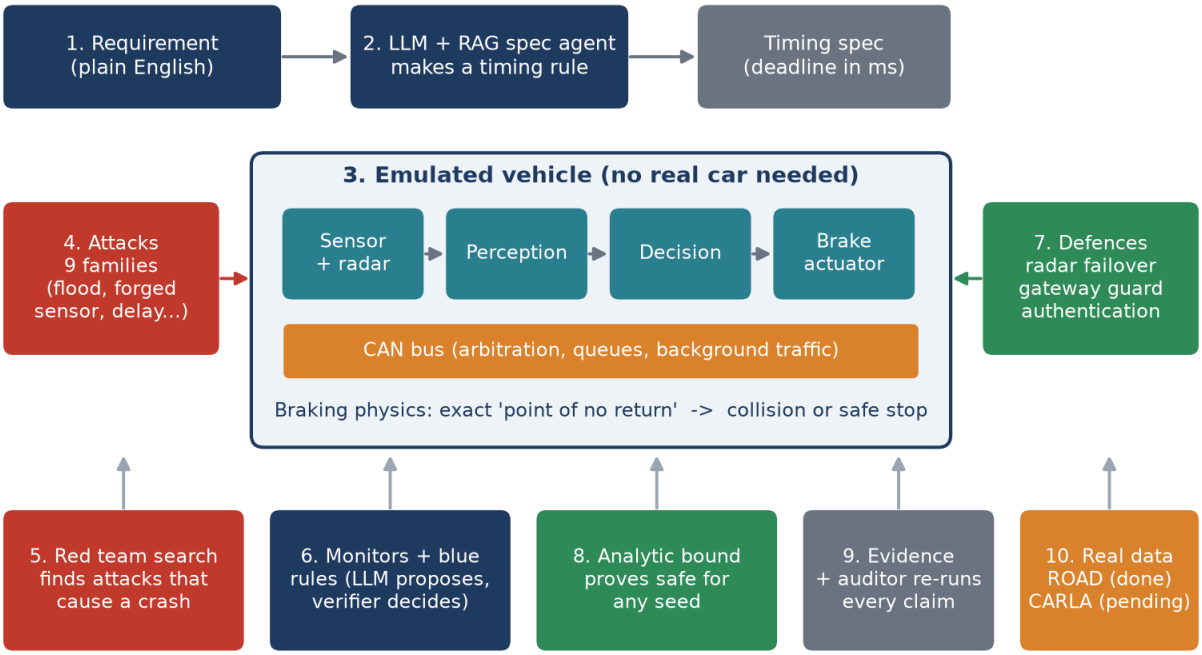
Updated on 15 October 2026 from the repository on branch `rebuild`. Every number in this document comes from a logged run in `experiments/RESULTS.md`. Things that are assumed, unfinished or not proven are said plainly.

1. What is this project, in one minute?

Modern cars are computers on wheels. Their parts (sensors, brake computer, radar, infotainment) talk to each other over a shared wire called the **CAN bus**. That bus has no passwords: any device plugged into it can send messages that look real. An attacker who gets onto the bus can **delay** the brake message, **flood** the wire so nothing else gets through, or **forge** the distance the sensor reports so the car thinks the road is clear.

The important question is not only *'was an attack detected?'* but *'was it detected, or stopped, **in time** to avoid a crash?'* Every crash scenario has a **point of no return**: the last moment at which braking can still save you. After that, nothing can.

Our idea: build a safe, repeatable test bench (an emulated car), attack it in many ways, measure whether defences act before the point of no return, add defences that actually remove crashes, and **prove** (for our model) how long the brake can be delayed by a given attacker.



Everything above is reproducible with one command and checked by 484 automated tests.

Figure 1. The whole project on one page. Numbers match the sections below.

2. The emulated car (part 3)

We cannot attack a real car, so we built a small **discrete-event emulator**: a clock that jumps from one event to the next, in whole microseconds, so a run with the same seed always gives the same answer. It has three layers.

2a. The CAN bus

Frames compete for the wire. When the bus is free, the frame with the **lowest ID wins** (that is how real CAN works), and nobody can interrupt a frame in progress. A frame's length uses the worst-case bit-stuffing formula from the CAN literature, at 500 kbit/s. The background traffic comes from the message catalogue in `Knowledge_base/can_messages.json`.

```
# sdv/bus/can_bus.py
def frame_bits(dlc: int) -> int:
    return 8 * dlc + 47 + (34 + 8 * dlc - 1) // 4      # worst case; 8 data bytes = 135 bits = 270 us

# when the bus is idle, the pending frame with the lowest identifier wins arbitration
winner = min(self._pending, key=lambda item: (item[1].can_id, item[0]))
```

2b. The brake chain

Four steps pass messages over the bus: **sensor** (sends distance and speed every 10 ms) to **perception** (raises a hazard when the reported time-to-collision drops below 2 s) to **decision** to the **brake actuator**. The compute time of each ECU (25, 15 and 25 ms on average, with random jitter and a CPU-load factor) is **assumed**; the delays through the bus are not assumed, they emerge from queues and arbitration.

2c. The braking physics and the point of no return

A short formula gives the exact answer: the car keeps its speed until the brake takes effect, then braking ramps up to its maximum. The largest delay that still avoids the obstacle is the point of no return.

```
# sdv/plant/longitudinal.py
def latest_safe_latency_s(v0, d0, decel, ramp_s):
    """Point of no return. Negative means a collision is unavoidable even at zero latency."""
    return (d0 - braking_distance(v0, decel, ramp_s)) / v0
```

Example: at 60 km/h with the obstacle 24 m ahead on a dry road, the car can wait about 353 ms before braking; after that it hits the obstacle.

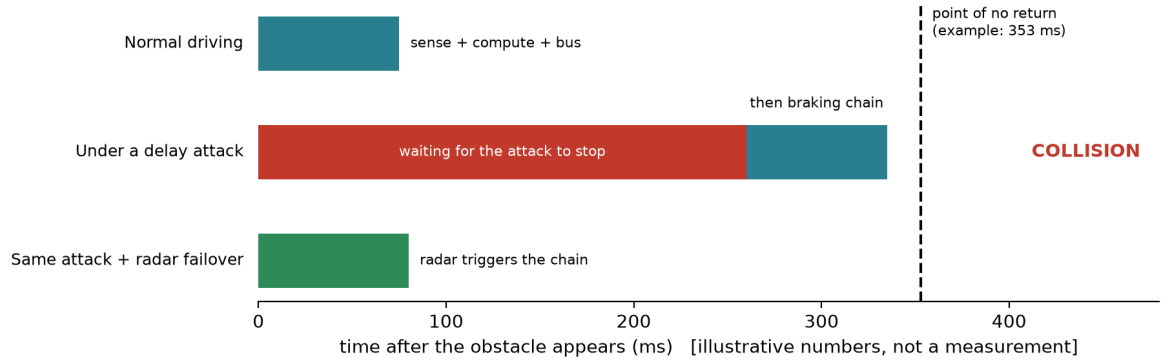


Figure 2. Why timing matters (illustrative numbers). A delay attack pushes the brake past the point of no return; a second, independent sensor lets the chain start without waiting.

3. The attacks (part 4)

Each attack family is a small class with a declared **capability** (what access the attacker needs) and **parameter bounds** (how strong it may be), so a search can explore them fairly.

Family	What the attacker does	Access needed
DoS flood, priority abuse, low-and-slow	Sends many high-priority frames so real ones wait	a node on the bus
Selective suppression	Drops some of the sensor's frames	compromised ECU or gateway
Gateway delay, jitter injection	Holds sensor frames back (fixed or random time)	compromised gateway
Sensor drift	Slowly raises the reported distance	compromised gateway
Masquerade	Replaces the sensor's frames with forged ones (constant extra distance)	compromised gateway
Dual masquerade	Forges the sensor **and** the radar the same way	compromised gateway

```
# sdv/attacks/library.py  (masquerade: forged frames at the legitimate period)
def flt(frame, now):
    if start <= now < end and frame.can_id == SENSOR_ID and frame.data.get("distance_m") is not None:
        frame.data["distance_m"] += self.bias_m          # the lie
        frame.data["forged"], frame.data["holds_key"] = True, self.holds_key
        return frame, int(self.phase_shift_ms * 1000)
    return frame, 0
```

Hazard volume is the share of an attack family's parameter space that causes a collision, found by sampling uniformly. Without any defence: masquerade 45%, DoS flood 28%, gateway delay 25%, suppression 4%, jitter 3%.

4. Detecting attacks in time (part 6)

Monitors watch the bus (they see frames, never the attacker's label) and raise an alarm: a frame-rate detector, a plausibility check, a deadline monitor, and a fused combination. All learn what 'normal' looks like from attack-free runs only. The key measure is borrowed from cyber-physical-systems research (the *detection deadline* idea of Akowuah and Kong): the **detection margin** is how much time was left between the alarm and the point of no return.

```
# detection margin = (latest safe latency) - (time the alarm came + 20 ms for the response)
# positive: the alarm was early enough to act; negative: too late
```

- **Main finding:** usual scores (recall, F1) make detectors look good, but the best fused detector flags only about 63% of attack-caused crashes *in time*.
- **A concrete blind spot:** a forged sensor that adds a constant distance, starting before the obstacle appears, looks perfectly self-consistent, so single-sensor checks miss it.

5. Searching for dangerous attacks (part 5)

The **red team** looks for attack parameters that cause a crash. We compared random search, grid search, Q-learning, an evolution strategy, Bayesian optimisation, and an LLM that suggests where to start (grounded in the knowledge base). **Honest result:** the clever methods were no better than random sampling in this small space; the LLM helped only where it named the real mechanism.

6. Where the LLM and RAG are used (part 2 and part 6)

Spec agent. A requirement in plain English becomes a machine-checkable rule. RAG looks up real signal names (the official COVESA vehicle-signal catalogue, 1,382 signals) so the model cannot invent them; a validator repairs bad output.

```
Requirement : "Brake within 100 ms if an obstacle is detected."
Timing spec : G(obstacle_detected -> F[0,100 ms] brake_applied)      # signal temporal logic, deadline 100 ms
```

Blue agent. After the red team finds an attack the monitors missed, an LLM proposes a new monitor rule, but a deterministic **verifier** decides whether it is accepted (at most 2% false alarms on normal runs, and at least one missed hazard now caught in time).

```
# a verified rule: if the sensor and the radar disagree about distance by more than 1 m, alarm
{'type': 'cross_check', 'id_a': '0x2A0', 'id_b': '0x2B0', 'field': 'distance_m',
 'tol': 1.0, 'max_skew_ms': 15.0}
```

Honest result: random rule proposals plus the same verifier did about as well as the LLM, so the verifier, not the LLM, does the work. The LLM measurably helps in one place: turning English into specs with the right signal names (retrieval raises correct-signal matching from about 6% to about 85% on our benchmark).

7. Defences that remove crashes (part 7)

Three defences, each added to the emulated car and each measured against every attack family.

7a. Radar failover (experiment E9)

Perception also reads an independent radar and brakes if **either** channel says the obstacle is close. A forged sensor alone can no longer hide the road.

```
# sdv/system/brake_chain.py
if frame.data.get("obstacle") and not self._detected:
    ttc = self._reported_ttc(frame)                    # distance / speed
    if ttc is not None and ttc <= self.params.ttc_trigger_s:
        self._start_chain(frame, channel)              # channel is "sensor" OR "radar"
```

7b. Gateway guard (experiment E12)

A gateway between the untrusted side (the OBD port, infotainment) and the safety bus lets through only whitelisted message IDs, at a limited rate. A flood from that side never reaches the safety bus.

```
# sdv/bus/guard.py
if frame.src != UNTRUSTED_SRC:
    return frame, 0                        # only frames from the untrusted side are checked
cycle_ms = self.allowed.get(frame.can_id)
if cycle_ms is None:
    self.blocked += 1
    return None, 0                         # ID not allowed from that side: dropped before arbitration
```

7c. Message authentication plus a fail-safe watchdog (experiment E12)

Frames carry a signature (idealised SecOC-style MAC). A gateway that changes a frame invalidates it and it is rejected. If neither the sensor nor the radar delivers a valid frame for 30 ms, the car brakes anyway (fail-safe).

```
# sdv/system/brake_chain.py
if frame.data.get("forged") and not frame.data.get("holds_key"):
    self.rejected_frames += 1              # MAC verification fails; the frame is discarded
    return
```

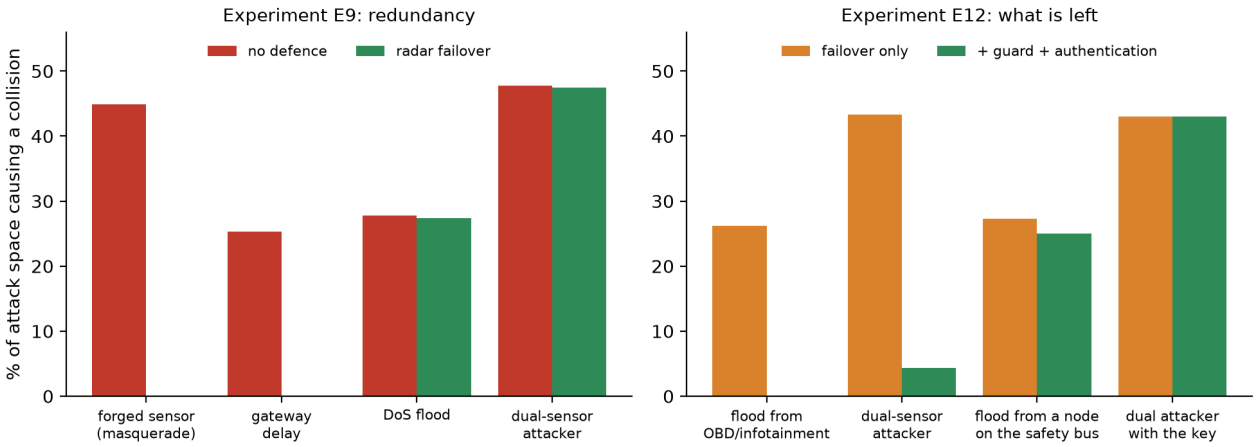


Figure 3. Measured results (hazard volume = share of the attack space that causes a collision). Left: radar failover. Right: after adding the gateway guard and authentication.

Attack	radar failover only	all three defences
Flood from the untrusted side (OBD/infotainment)	26.2%	0.0%
Dual-sensor attacker (forges both channels)	43.3%	4.3% (3.0-6.3%)
Flood from a node already on the safety bus	27.3%	25.0% (still open)
Dual-sensor attacker who holds the signing key	43.0%	43.0% (still open)

- **What is left, plainly:** a flood from a node already on the safety bus, and an attacker who holds the key. Authentication moves trust to key management; the guard moves it to the network design.
- **Costs we measured:** authentication adds about 1 ms to normal braking. It stops a single forged channel from causing unwanted braking, but if both channels are corrupted the watchdog now brakes with nothing in the way (an availability cost). Radar-only voting without authentication lets a radar forger cause unwanted braking.
- **Assumptions:** the MAC is idealised (key management and freshness not modelled); the 30 ms watchdog and 0.5 ms verification time are assumed.

8. Proving a safe limit: the analytic bound (part 8)

Testing shows what happened in the runs we tried. A **bound** says what can *never* be exceeded. For a scenario and an attack, `sdv/analysis/bounds.py` computes the worst-case time from the obstacle appearing to the brake acting, using classic CAN response-time analysis plus a model of each attack. If that time is below the point of no return, **no random seed can cause a crash**.

```
# L = T_trig + R_channel + P + R_detect + D + R_cmd + A
#   T_trig   when the first frame reporting 'close' can be sent (a forged distance delays it)
#   R_*      worst-case CAN frame response times (queues, blocking, the attacker's frames)
#   P, D, A  worst-case compute times of perception, decision and the brake actuator
proven_safe = latency_bound_ms(scenario, attack, chain).total_ms <= latest_safe_latency_ms(scenario)
```

- **It held in every test:** 0 violations in more than 31,000 simulated runs per configuration set, and 0 of 260 adversarial searches (a search that tries to *break* the bound) found a counter-example.
- **The adversarial search found three real bugs** in my first versions (floods that leave a queue behind). They are fixed and each has a regression test.
- **It is conservative:** the median bound is about twice the observed latency, so 'unknown' means 'the bound is too coarse', not 'unsafe'.
- **It is a proof for the model** (assumed compute times, the emulated bus), not for a real vehicle.

9. Evidence you can re-check (part 9)

The pipeline writes an **evidence bundle**: requirement, spec, the attack that caused a crash, the threat it maps to, the fix, and the outcome under every defence. An **auditor** then re-runs every claim from scratch (20 checks) and fails if any number was changed. We tested this by tampering with latencies, bounds, rules and citations; each is caught. It is checked consistency on an emulation, not a certified safety case.

10. Real data: the ROAD dataset (part 10)

To avoid relying only on our emulator, we replayed 13 **real** attack recordings from one vehicle (Oak Ridge National Laboratory). Detectors learned only from normal driving. Results: a redundancy cross-check caught real single-signal forgeries within about 4 ms with zero false alarms on held-out normal data; forgeries that stay inside the normal range (reverse-light attacks, 0/6) were missed by everything; a simple frame-rate detector was unusable (about 409 false alarms per hour). ROAD has no braking and no dual-sensor attacker, so it validates the detectors only, not crashes or margins.

11. Where things stand, and what is not proven

Item	Status
Emulator, attacks, monitors, search, spec agent, blue loop	Done, reproducible with one command (also rerun in the cloud), 484 tests pass
Defences (failover, guard, authentication) and the analytic bound	Done, validated by simulation and adversarial search
Evidence chain and auditor	Done, tamper-tested
Real attack data (ROAD)	Done (13 captures, one vehicle): detectors, and the authentication + watchdog defence replayed on the real attacks
CARLA check of the braking physics	Done (4 runs on CARLA 0.9.15): the plant's structure agrees to about 11 ms once calibrated; my default braking numbers are optimistic
Research paper	Not written yet (planned after the CARLA step)

Limits to state honestly in any paper:

- Everything about vehicles is emulated, with **assumed** ECU timings (a sensitivity test shows the conclusions survive 0.5x to 2x, but absolute percentages should not be quoted as real-world rates).
- Novelty is incremental: the 'detection deadline' idea already exists; our contribution is applying and extending it to bus-level CAN attacks, the defences, the proof-style bound, and the open, audited artifact. I read one close related paper (Berdich and Groza) only as an abstract.
- Two attacker types are still not stopped: a flood from a node on the safety bus, and a key holder.
- The LLM did not beat random proposals for monitor rules; this is reported as a negative result.

12. How to run it

```
python -m pytest tests -q                                  # 484 tests
python -m sdv --requirement "Brake within 100 ms if an obstacle is detected." --blue enumerate
python experiments/reproduce_all.py                        # regenerates every result table (about 40 min)
python experiments/e12_defences.py --scenarios 6 --samples 100   # one experiment
```

Key folders: `sdv/` (the code), `experiments/` (E1-E13 and `RESULTS.md`), `docs/` (novelty analysis, CARLA notes), `tests/`.

Glossary

Term	Meaning
CAN bus	The shared wire that car computers use to talk; no built-in security
ECU	A small computer in the car (sensor, brake controller, ...)
Point of no return	The latest brake delay that still avoids the obstacle
Detection margin / deadline	Time between an alarm and the point of no return
Hazard volume	Share of an attack's parameter space that causes a crash
RAG	Looking facts up in a knowledge base before the LLM answers
STL	Signal temporal logic: a formula such as 'brake within 100 ms'
MAC / SecOC	A signature on each message so forgeries are rejected
Red / blue team	Red looks for attacks; blue builds defences